



Faculty Of Computers & Data Science

GRADUATION PROJECT REPORT

Reverie

A Data Science Automation Tool

G45

Supervisor: Dr. Yasser Fakhry

Authors:

Ahmed Mohamed Amin 210101074

Abdelrahman Magdy 210101071

Omar Khaled 210101073

Ali Yaqout 20221374071

Amr Samy 20221060922

Mahmoud Abdelghany 20221460236

Abdullah Nasser 20221041225

Kareem Ahmed 20221460156

Contents

1. Abstract	2
2. Team	3
3. Introduction	6
○ 3.1 Project Overview	6
○ 3.2 Problem Statement	6
○ 3.3 Project Scope and Objectives	7
○ 3.4 Significance of the Project	8
○ 3.5 Target Audience	8
○ 3.6 Expected Outcomes	9
4. Literature Review	10
5. Requirements and Methodology	15
○ 5.1 Functional Requirements	15
○ 5.2 Non-Functional Requirements	17
○ 5.3 Dependency Table	18
○ 5.4 Development Model	19
6. Implementation	23
○ 6.1 System Architecture	23
○ 6.2 Unified Modeling Language	26
7. Testing Plans	29
○ 7.1 Unit Testing	29
○ 7.2 Integration Testing	30
○ 7.3 User Testing	31
8. Coding Practices	33
9. Conclusion	35
10. References	36

Abstract

Data science has emerged as a cornerstone for decision-making across diverse industries, yet its complexity often acts as a barrier for non-programmers. Recognizing this challenge, our team of eight has undertaken the development of a data science automation software as our graduation project. The primary objective is to create a user-friendly platform that empowers users, regardless of their technical background, to perform essential data science tasks seamlessly, from data cleaning to basic predictive modeling.

The proposed software will feature an intuitive interface designed to simplify operations such as data cleaning, visualization, and basic modeling, all without requiring users to write code. By eliminating the need for programming skills, the platform aspires to democratize data science, making it accessible to a broader audience. The project will implement a modular approach, with each feature tested through iterative development to ensure ease of use and functionality.

Our vision is to bridge the gap between technical complexity and practical usability, enabling professionals from various fields to harness the power of data science in their work. Initial testing and user feedback will be used to refine the platform's features and functionality.

This project represents an exciting opportunity to contribute to the growing demand for accessible data science tools, and it paves the way for future innovation in this domain. By simplifying the data science workflow, we aim to empower a new generation of users to make data-driven decisions with minimal technical expertise.

TEAM

Our team consists of 8 dedicated members who worked together to develop a comprehensive data science automation software using **Electron**. We divided ourselves into two main sections **Frontend** and **Backend** to ensure an efficient and organized approach to the project. Each team member played a crucial role in contributing to the success of the project.

Backend Team

The Backend team was responsible for developing the core functionality of the application, including data processing, machine learning model integration, and communication with the frontend. The team ensured that the software could handle large datasets, implement various data science tasks (such as data cleaning, modeling, and evaluation), and maintain smooth interaction with the frontend interface.

Ahmed Mohamed Amin

Ahmed was one of the key contributors to the backend development. He focused on building the core data science functionality, including data import/export, and integrating machine learning models for classification and regression tasks. His strong programming skills and attention to detail ensured that the backend system operated efficiently and seamlessly.

Abdelrahman Magdy

Abdelrahman worked alongside Ahmed in implementing data processing functionalities, with a particular focus on **data cleaning** and **normalization** techniques. He integrated various data cleaning libraries and created efficient data pipelines to prepare data for analysis and modeling. His work was crucial in ensuring that the data was cleaned and ready for modeling, which is one of the fundamental steps in data science projects.

Abdullah Nasser

Abdullah's primary focus was on **data modeling** and evaluating machine learning models. He implemented the algorithms for training models and evaluating them using standard metrics such as accuracy, precision, and recall. Abdullah's contributions were instrumental in ensuring that the software could handle different types of machine learning models and evaluate them effectively for data science tasks.

Kareem Mohamed

Kareem contributed significantly to the backend development by working on **data storage** and **API integration**. He helped design the system for managing datasets, ensuring that large datasets could be efficiently processed and stored. His expertise in building APIs allowed the

backend to communicate effectively with the frontend, ensuring seamless data flow and interaction across the application.

Frontend Team

The Frontend team was responsible for developing the **user interface (UI)** and **user experience (UX)** of the software. They ensured that the software was intuitive, user-friendly, and visually appealing. The team focused on creating an easy-to-navigate interface that allowed users to seamlessly perform data science tasks like data import/export, data cleaning, and model training without needing any prior technical knowledge.

Amr Samy

Amr led the **UI/UX design** for the frontend. He worked closely with the backend team to ensure that the user interface was well-integrated with the system. Amr designed the layout, navigation, and visual elements, focusing on simplicity and user-centered design principles. His contributions were essential in making sure that the software was not only functional but also accessible and intuitive for users.

Omar Khaled

Omar focused on the **frontend development** using **Electron**. He was responsible for implementing the layout and functionality of the various panels and features, such as data import/export, visualization, and model training. His work ensured that the frontend was responsive and interacted effectively with the backend services. Omar's knowledge of web technologies was essential in bringing the visual design to life and ensuring smooth user interactions.

Mahmoud Abdelghany

Mahmoud worked on implementing the interactive **data visualization** features. He developed tools that allowed users to view and interact with their data through charts and graphs. Mahmoud integrated popular JavaScript libraries to visualize the processed data, enabling users to gain insights through customizable visualizations. His focus on making visualizations both functional and aesthetically pleasing was crucial for the success of the project.

Ali Yaqout

Ali's contributions focused on the overall **frontend structure** and ensuring smooth interaction between the **UI elements** and the **backend functionalities**. He worked closely with Omar and Mahmoud to ensure that the software provided an intuitive and seamless user experience, particularly in managing user inputs and displaying the results. His attention to detail in designing user flows and interactions helped in making the software both engaging and user-friendly.

Collaboration and Teamwork

Throughout the development process, the frontend and backend teams collaborated closely to ensure the seamless integration of all features. Regular meetings and feedback sessions allowed the teams to align their progress and address any challenges that arose. Despite the different roles, every team member contributed to the overall success of the project by sharing their expertise and supporting each other.

The successful completion of this data science automation software is a testament to the hard work, dedication, and collaborative efforts of the entire team. We are proud to have worked together on this project, bringing together our diverse skill sets to create a tool that will empower users to automate and streamline their data science tasks.

INTRODUCTION

Project Overview

Data science is a multidisciplinary field that combines statistics, computational science, research techniques, data processing, visualization, algorithms, and systems to derive insights and knowledge from both structured and unstructured data, which may be noisy [1]. In today's data-driven world, the ability to analyze and interpret large volumes of data has become essential for organizations to remain competitive. Data science, with its powerful tools and techniques, plays a pivotal role in transforming raw data into actionable insights. From predicting market trends to identifying inefficiencies in business processes, the applications of data science are far-reaching. However, the complexity of many existing data science tools and the steep learning curve required to use them effectively can pose significant barriers, especially for individuals without programming experience.

This project aims to develop an **Automation software tool for Data Science** designed to address this very challenge. Our goal is to create a user-friendly platform that empowers users, regardless of their technical background, to perform key data science tasks with ease. The software will simplify the process of data analysis by offering an intuitive graphical user interface (GUI) that eliminates the need for users to write code. By providing a seamless experience for data cleaning, visualization, and basic modeling, the platform will make data science accessible to a broader audience, including professionals in fields such as healthcare, education, finance, marketing, and more.

Problem Statement

Despite the increasing availability of data across industries, many individuals are still unable to effectively harness the power of this data due to the technical expertise required for data analysis. While data science tools like Python, R, and MATLAB are highly effective, they require a level of programming knowledge that many professionals do not possess. This gap between the availability of data and the ability to analyze it is particularly pronounced in industries where data science could provide significant value but where employees lack the necessary technical skills.

The difficulty in accessing advanced data science tools hinders decision-making processes, as

valuable insights remain untapped. Non-technical users are often forced to rely on external data scientists or consultants to conduct analyses, which can be both time-consuming and costly. The lack of user-friendly tools that bridge this gap creates a pressing need for an accessible data science solution—one that allows users with limited technical experience to carry out data science tasks on their own.

Reverie is designed to fill this gap, offering an intuitive and powerful platform that empowers non-technical users to perform data analysis without requiring deep programming expertise.

Project Scope and Objectives

This project seeks to fill the gap by developing an intuitive software platform capable of simplifying and automating essential data science tasks. Reverie will feature a set of functionalities that allow users to work with their data in a straightforward manner, including:

- **Data Cleaning:** A crucial step in the data science pipeline, where users can easily handle missing values, outliers, and errors in their datasets. With just a few clicks, users will be able to preprocess their data and prepare it for analysis, without needing to write complex code.
- **Data Visualization:** This feature will allow users to create various types of visualizations—such as bar charts, scatter plots, and histograms—by simply dragging and dropping their data into a user-friendly interface. Users can customize the appearance and presentation of their visualizations without requiring any knowledge of coding or data visualization libraries.
- **Basic Modeling:** For users interested in performing predictive analysis, the platform will offer a range of basic machine learning algorithms, including regression and classification models. The software will walk users through the process of training and testing these models, providing them with the ability to make data-driven predictions and insights without the need for any technical expertise.

The software will be designed with simplicity and usability in mind, ensuring that even users with minimal technical backgrounds can leverage its features effectively. By providing an accessible solution to data analysis, this tool will help users make informed decisions based on

data, whether they are working in a corporate setting, a small business, or a non-profit organization.

Significance of the Project

The importance of this project lies in its potential to democratize access to data science tools. As data-driven decision-making becomes more critical, the need for tools that empower non-technical users to analyze and interpret data is greater than ever. By eliminating the need for specialized skills, this software will make data science more accessible, allowing a wider range of professionals to harness the power of their data.

Moreover, this platform will enable users to conduct basic data analyses without the need for external consultants or data scientists, reducing costs and improving the speed at which decisions can be made. This will empower professionals in various industries—including healthcare, education, marketing, and finance—to make better-informed, data-driven decisions that can lead to improved outcomes and greater efficiency.

In addition, the software's modular design will allow it to be easily updated with new features or tools as the field of data science evolves, ensuring that it remains relevant and useful in the long term. This adaptability also provides room for further innovations in the future, potentially incorporating more advanced data science techniques or machine learning models as the platform matures.

Target Audience

The target audience for this software includes professionals and organizations that recognize the value of data science but are not equipped with the resources or expertise to fully utilize existing tools. This includes business analysts, marketing professionals, HR managers, healthcare professionals, and educators who deal with data regularly but lack the technical skills to perform complex analyses. Additionally, small businesses and non-profit organizations that may not have access to full-time data scientists will benefit from this tool, as it provides a cost-effective alternative to hiring external experts.

Expected Outcomes

The expected outcome of this project is the development of a fully functional, user-friendly software platform that simplifies the core tasks of data science. The platform will enable users to:

- Clean and preprocess their data without needing programming knowledge.
- Visualize their data and generate actionable insights.
- Build basic predictive models to make data-driven decisions.

By the end of the project, we aim to provide a tool that is not only functional but also intuitive and accessible, contributing to the democratization of data science and empowering a wide range of professionals to make informed, data-driven decisions.

LITERATURE REVIEW

The field of data science has witnessed rapid advancements over the past decade, with an increasing number of tools, techniques, and methodologies being developed to handle various aspects of data analysis, machine learning, and predictive modeling [2]. However, while the growth of data science is undeniable, it has also introduced a significant challenge: the technical complexity of existing tools often limits their accessibility to non-technical users. In this section, we review the relevant literature that informs the development of our data science automation software, focusing on user accessibility in data science tools, automated data analysis, and the role of software development methodologies in building accessible solutions.

Challenges in Data Science Accessibility

One of the key challenges in data science today is the **barrier to entry** posed by the complexity of programming languages and software tools [3]. While languages like **Python** and **R** have become ubiquitous in the data science community due to their extensive libraries and frameworks, they require a certain level of proficiency in programming, which most professionals in fields like healthcare, business, and education do not possess. As noted by **LaVille et al. (2011)**, a significant gap exists between the growing demand for data-driven insights and the lack of technical expertise in many sectors. This issue creates a situation where organizations have access to valuable data but lack the in-house resources or skills to analyze it effectively [4].

To address this issue, **low-code** and **no-code** tools have been developed. These platforms enable users to perform data analysis and build machine learning models with minimal or no programming knowledge. Notable examples include tools like **Power BI**, **Tableau**, and **RapidMiner**, which allow users to create data visualizations and build predictive models using drag-and-drop interfaces [5]. However, while these tools are user-friendly, they often lack the flexibility and depth of traditional programming-based solutions. It's highlighted that many of these tools are limited in terms of customization, making them less suitable for advanced data science tasks [6]. Moreover, they are often designed for specific domains, making them less adaptable for users across different industries.

The Role of Automation in Data Science

Automation has become a key focus in the field of data science, particularly in areas such as **data cleaning**, **feature engineering**, and **model selection** [7]. The automation of routine tasks can significantly reduce the time and effort required to prepare data for analysis, which is often the most time-consuming aspect of a data science project. According to **Jhonathan David (2023)**, automating repetitive tasks like data preprocessing allows data scientists to focus on more complex tasks, such as modeling and interpretation [8]. However, the integration of automation in tools that are accessible to non-technical users remains an area of active research.

One of the prominent solutions to address the challenges faced by non-technical users in the field of data science is **AutoML** (Automated Machine Learning). AutoML refers to a set of techniques that automate the process of selecting, training, and tuning machine learning models, making machine learning more accessible to people with limited expertise in the field. By removing the need for deep knowledge of algorithms, model selection, and parameter tuning, AutoML provides a valuable pathway for non-experts to build accurate predictive models.

Tools like Google AutoML and Auto-sklearn

Two notable AutoML tools are **Google AutoML** and **Auto-sklearn**, both of which have made significant strides in simplifying the machine learning pipeline for non-technical users:

1. **Google AutoML:**

Google AutoML offers a suite of machine learning services that enable users to create custom models without requiring extensive coding knowledge. This tool leverages Google's deep learning expertise to automatically select the best model and hyperparameters for a given dataset. It includes features like image classification, natural language processing, and tabular data analysis. The main appeal of Google AutoML lies in its user-friendly interface, which allows users to upload their data and receive a trained model, all while abstracting away the complexities typically associated with machine learning. However, while the process is simplified, understanding the underlying concepts (such as the type of data being used, the nature of the model's output, or how hyperparameters are optimized) may still pose a challenge for complete beginners.

2. **Auto-sklearn:**

Auto-sklearn is an open-source AutoML tool built on top of the popular Python library scikit-learn. It automates the selection and optimization of machine learning models and their hyperparameters. Auto-sklearn performs model selection, feature preprocessing, and hyperparameter tuning by automatically trying different algorithms and configurations. It uses Bayesian optimization to fine-tune hyperparameters for improved performance.

While Auto-sklearn does an excellent job of automating the model-building process, it still requires some familiarity with Python and basic machine learning concepts to effectively use it. For instance, users need to be comfortable with the scikit-learn framework and be able to interpret the results provided by Auto-sklearn.

Limitations for Novices

While tools like **Google AutoML** and **Auto-sklearn** significantly reduce the complexity of building machine learning models, they still have limitations that may pose barriers for absolute beginners:

- **Understanding the Model's Output:** Although these tools automate the model-building process, they often don't provide a deep explanation of how the model works or why certain decisions were made during the process. This can make it difficult for non-experts to interpret the results or explain them in a business context.
- **Data Preparation:** Despite the ease of use, users still need to understand basic concepts such as feature engineering, data cleaning, and preprocessing. These are often crucial for achieving the best results, and users without technical knowledge may struggle with preparing their data effectively.
- **Choosing Between Models:** While these tools automate the selection of machine learning algorithms, novice users may still face difficulties in understanding why one algorithm is chosen over another. For example, in cases where multiple models are suggested, a user may not know which one is best suited for their data and problem.

Recent studies, such as **Kush Patel (2024)**, have focused on developing fully automated data science platforms that can handle the end-to-end process of data analysis—from data

preprocessing to model deployment—without requiring any technical expertise. These platforms aim to democratize data science by making it accessible to users in industries like business analytics, healthcare, and social sciences [9].

Data Science in Non-Technical Fields

The demand for accessible data science tools is particularly high in **non-technical fields** where data-driven decision-making is becoming increasingly essential [10]. In healthcare, for instance, the rise of electronic health records (EHR) has created vast amounts of data that can provide valuable insights into patient care, treatment effectiveness, and operational efficiency. However, healthcare professionals often lack the programming skills required to analyze this data effectively. A study by **Pushpa et al. (2024)** found that while there is a growing interest in using data science techniques in healthcare, the lack of user-friendly tools and training programs is a significant barrier to the widespread adoption of data science [11].

Similarly, in the field of education, institutions are gathering large volumes of student performance data, but the lack of accessible tools makes it difficult for educators to use this data to improve learning outcomes. It's argued that accessible data science tools could help educators make more informed decisions about curriculum design, resource allocation, and student support services [12].

Power BI and Its Role in Democratizing Data Science

Power BI, developed by Microsoft, is a widely used business intelligence tool that enables users to create interactive visualizations and dashboards without requiring programming knowledge. Its user-friendly interface and drag-and-drop functionality make it accessible to non-technical users, allowing them to easily analyze and visualize data from various sources, including Excel, SQL databases, and web services. Power BI also provides a **Power Query** tool for data cleaning and transformation, further simplifying the process.

However, a limitation of Power BI and similar tools like Tableau and QlikView is that they are not open source [6]. This means that users do not have access to the underlying source code and are restricted in terms of customization and flexibility. This proprietary nature of the tools can

limit the ability to tailor solutions for specific use cases or to integrate them with custom-built systems. Furthermore, the cost of licenses for full access to advanced features can be prohibitive for some users, particularly in small businesses or educational institutions.

While these tools are useful for visualizations and reporting, they fall short for more advanced data science tasks such as custom data manipulation or complex machine learning model training. This highlights the need for more specialized tools that can support both business intelligence and advanced data science, a gap that our software aims to fill.

REQUIREMENTS & METHODOLOGY

In this section, we outline the functional and non-functional requirements of our data science automation software and describe the incremental software development methodology used to ensure the project's success.

Functional Requirements

The functional requirements define the specific capabilities and features the software must provide. These include:

1. Data Import and Export

- Allow users to upload datasets in common formats such as CSV, Excel, and JSON.
- Support export of processed datasets and visualizations in user-selected formats.

2. Data Cleaning

- Provide an interface for identifying and handling missing values, outliers, and duplicate records.
- Offer tools for filtering, sorting, and reformatting data.

3. Data Transformation

- Enable users to apply transformations such as scaling, normalization, and encoding categorical variables.
- Support creating new columns through arithmetic or logical operations.

4. Data Visualization

- Include built-in templates for creating bar charts, line graphs, scatter plots, and heatmaps.
- Allow customization of visualizations with labels, legends, and color schemes.

- Support exporting visualizations in high-quality formats such as PNG or PDF.

5. Basic Modeling

- Enable users to apply pre-built models for classification, regression, and clustering tasks.
- Provide model evaluation metrics such as accuracy, precision, recall, and F1-score.

6. Presets and Templates

- Include pre-configured settings for common workflows, such as data cleaning pipelines or visualization layouts.
- Allow users to save their own presets for reuse in future projects.

7. Settings Page

- Allow users to configure software preferences, such as theme (dark/light mode), language, and default file paths.
- Provide options for managing user accounts and saving personalized settings.

8. Real-Time Insights and Suggestion

- Provide real-time feedback on dataset quality, including missing values, outliers, and inconsistencies.
- Offer suggestions for appropriate visualizations or models based on dataset characteristics.

9. No-Code Interface

10. Help and Documentation

Non-Functional Requirements

The non-functional requirements focus on the performance and operational quality of the software:

1. Usability

- The interface should be user-friendly, ensuring ease of navigation for non-technical users.

2. Scalability

- The software should handle datasets of varying sizes, from small to moderately large.

3. Performance

- The system should process datasets and generate results within reasonable timeframes.

4. Compatibility

- Ensure compatibility across major operating systems like Windows, macOS, and Linux.

5. Security

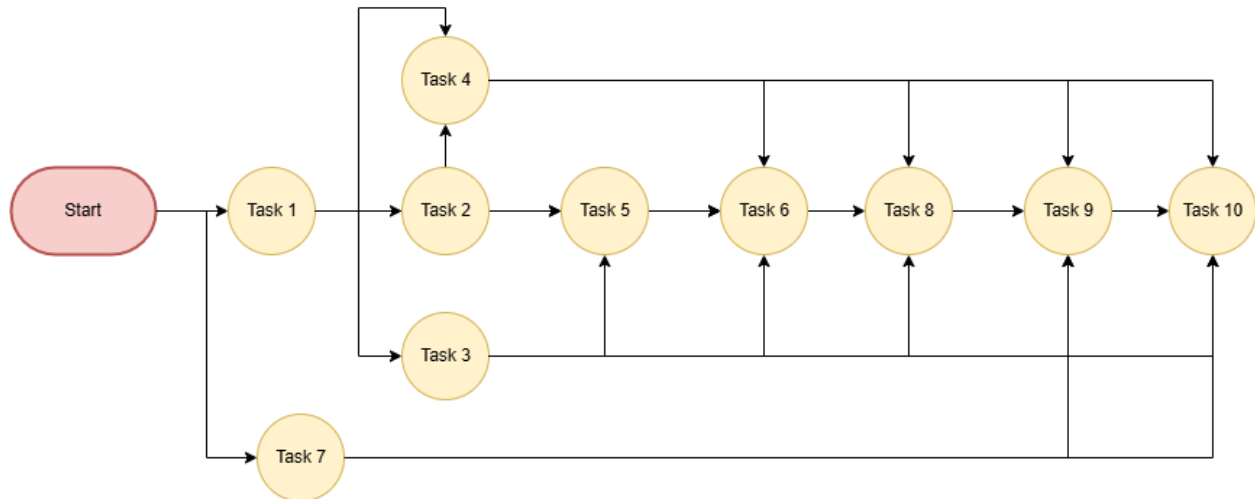
- Protect user data during import/export operations and ensure compliance with basic data privacy standards.

Dependency Table

Task	Hours	Dependency
Task 1	4	
Task 2	8	T1
Task 3	10	T1
Task 4	12	T1, T2
Task 5	18	T1, T2, T3
Task 6	10	T1, T2, T3, T4, T5
Task 7	4	
Task 8	24	T1, T2, T3, T4, T5, T6
Task 9	10	All
Task 10	6	All

This dependency table outlines the relationships between key features of the software, highlighting the order and prerequisites required for their implementation. Each feature is mapped to the functionalities it depends on, ensuring seamless integration and logical workflow progression. For example, core features like "Data Import and Export" serve as the foundation for advanced capabilities such as "Data Cleaning," "Transformation," and "Visualization." Features like "Help and Documentation" and "No-Code Interface" rely on the implementation of all other functionalities to provide comprehensive user support and an intuitive experience. This table serves as a blueprint for development and integration planning.

Activity Flow Diagram



Development Model

For the development of Reverie, we adopted the **Incremental Software Development Model**. This model allowed us to break down the complex project into smaller, more manageable phases. Each phase or module was developed and delivered incrementally, ensuring that we could evaluate progress continuously and make improvements at each step. This approach was particularly beneficial for our graduation project, as it enabled us to maintain a clear and structured workflow while incorporating valuable feedback throughout the development process.

Why the Incremental Model?

The **Incremental Model** was selected due to its combination of both **linear** and **iterative development** characteristics, making it a highly effective approach for our project. Here's a more detailed rationale for choosing this model:

1. **Flexibility in Development:** The incremental approach allows for a gradual and flexible development process. Instead of attempting to build the entire system at once, we could focus on delivering key functionalities one at a time. This allowed us to address any issues early and adjust the direction as needed based on feedback.
2. **Early Delivery of Core Features:** By adopting the incremental approach, we prioritized the development of core features, such as **Data Import/Export** and **Data Cleaning**. These functionalities were essential for the system to be usable and effective. Once these foundational modules were completed and verified, we moved on to developing advanced features like **Data Modeling** and **Visualization**. This ensured that the software was functional at every stage of development, even as new features were being added.

3. **Feedback-Driven Improvements:** A major advantage of the incremental model is the opportunity to **receive and incorporate feedback** at the end of each iteration. After each phase, we presented the software to our supervisor, gathered their input, and used it to refine the existing features before moving forward. This iterative feedback loop was crucial in ensuring the software met the required standards and addressed all user needs.
4. **Risk Mitigation and Continuous Evaluation:** By breaking the project into smaller phases, we were able to mitigate risks associated with the development process. Each phase served as a mini-project, allowing us to evaluate progress, test the features, and identify potential issues early. This not only helped in ensuring high-quality output but also reduced the chances of costly mistakes that might arise from tackling the entire project at once.
5. **Smooth Transition Between Phases:** The incremental model allowed us to create a natural progression between different stages of development. We built on top of existing features, continually improving and expanding the software. As a result, we were able to maintain consistency across the project while gradually increasing the complexity of the system.

Phases of Development

The development of Reverie is divided into five key phases. This incremental approach allows us to focus on one area at a time, ensuring quality and continuous progress with feedback at each stage.

Phase 1: UI/UX and Frontend Development

In **Phase 1**, we focus on the **design and development of the user interface (UI) and user experience (UX)**. This phase is crucial because a user-friendly and intuitive interface is key to the success of the software.

We start by designing the **overall layout**, keeping the user in mind to ensure smooth navigation. The **frontend framework** is selected, and the UI elements, such as buttons, forms, and menus, are implemented. The **GUI** integrates seamlessly with the backend functionality, ensuring that users can interact with the software efficiently.

This phase also includes the initial design of the **workflow** for importing, cleaning, and visualizing data. The aim is to create a cohesive and visually appealing experience for the end user.

Phase 2: Data Cleaning and Normalization

Phase 2 is centered around implementing the **data cleaning** and **normalization** functionalities. This phase is vital because it ensures that the data is in a usable state for further analysis and modeling.

The **data cleaning tools** are developed to handle missing values, duplicates, and outliers. Different strategies for dealing with missing values (e.g., imputation or removal) and outliers are

integrated. **Data normalization** is also introduced in this phase, allowing users to standardize or scale their datasets as needed.

By the end of this phase, the software will have solid foundations for preparing data, making it ready for more complex tasks like visualization and modeling.

Phase 3: Data Visualization

In **Phase 3**, we focus on creating the **visualization module**. This module provides users with powerful and interactive tools to explore their data visually.

The goal of this phase is to develop various types of **interactive visualizations**, such as bar charts, line graphs, scatter plots, and heatmaps, with customizable options. Users should be able to **filter, zoom, and adjust** their charts to gain insights into the data.

The visualization module not only helps users interpret the data but also makes it easier to communicate findings, especially for non-technical stakeholders. The visualizations will be integrated into the GUI, allowing users to easily interact with them as they work through the software.

Phase 4: Data Modeling

Phase 4 is dedicated to implementing the **data modeling** features of the software. This phase allows users to apply various **machine learning algorithms** for tasks like **classification, regression, and clustering**.

Pre-built models such as **logistic regression, decision trees, k-nearest neighbors, and k-means clustering** are integrated into the software. The users can select the model type, configure the parameters, and run the models on their data.

This phase also includes the integration of **model evaluation metrics**, such as accuracy, precision, recall, and F1 score, to assess the performance of the models. The results of these models will be displayed within the GUI, enabling users to analyze model outcomes directly within the software.

Phase 5: Deployment

In **Phase 5**, the focus shifts to **final deployment**. This phase involves integrating all the modules developed in earlier phases and ensuring the software is ready for end users.

Once the software is fully integrated, we perform **final testing** to check for any issues, bugs, or performance bottlenecks. Optimization is performed to ensure that the software runs efficiently, especially when dealing with large datasets or complex computations.

After successful testing, the software is **deployed** for general use, and we provide **documentation** and **user manuals** to help users navigate and make the most of the software.

This phase ensures that the software is fully functional, reliable, and ready for real-world application.

Key Advantages of the Incremental Model

- **Continuous User Feedback:** Allowed early testing of functionalities by target users, enabling iterative refinement.
- **Reduced Risk:** Breaking the project into smaller increments minimized the risk of major failures during development.
- **Prioritized Features:** Critical features were implemented and tested early, ensuring a usable product even if additional modules faced delays.

IMPLEMENTATION

System Architecture

The system was designed using a modular architecture to enhance scalability, maintainability, and ease of development. The architecture includes the following core components:

1. Frontend (GUI):

- Built using **Electron.js**, the frontend provides a modern, cross-platform graphical user interface (GUI). This interface simplifies data science operations for users with little to no programming experience. Features like drag-and-drop dataset uploads, interactive wizards, and customizable workflows enhance usability and accessibility.
- **Why Electron.js?**
 1. **Cross-Platform Support:** Electron enables the creation of desktop applications that work seamlessly on Windows, macOS, and Linux, ensuring broad compatibility for end users.
 2. **Web Technology Stack:** It allows us to use familiar web technologies (HTML, CSS, and JavaScript) to build the interface, which simplifies development and enables rapid prototyping.
 3. **Customizability and Flexibility:** Unlike pre-built frameworks or tools with rigid templates, Electron offers the flexibility to design a highly customizable and interactive user interface tailored to the needs of non-technical users.
 4. **Community Support:** Electron has a large and active community, providing access to numerous plugins, libraries, and documentation, which facilitates development

2. Backend:

- Powered by **Django**, a Python web framework, the backend handles data processing, integration with machine learning libraries, and communication between the GUI and data science modules.
- **Why Django?**
 1. **Rapid Development:** Django provides built-in features such as authentication, ORM (Object-Relational Mapping), and admin interfaces, which accelerate development.
 2. **Scalability:** The framework's ability to handle large-scale projects makes it suitable for future enhancements and expansions of our platform.
 3. **Security:** Django includes robust security features, such as protection against SQL injection, cross-site scripting (XSS), and cross-site request forgery (CSRF).
 4. **Python Ecosystem Integration:** Given our reliance on Python libraries for data science, Django aligns perfectly with our overall technology stack.

3. AI Model Integration:

- Pre-trained AI models (e.g., GPT) were fine-tuned for data science-specific tasks such as dataset analysis, error detection, and model recommendations.
- Lightweight models optimized using frameworks like **ONNX (Open Neural Network Exchange)** ensured responsiveness and performance efficiency.

4. Visualization & Modeling:

- Python libraries such as **Pandas**, **NumPy**, and **Matplotlib** were integrated for data manipulation, analysis, and visualization. These libraries provided extensive functionalities for handling datasets and generating visual insights, including correlation matrices, ROC curves, and feature importance plots.

- **Why?**

1. **Pandas:** Provides powerful tools for data manipulation and analysis, making it ideal for handling structured data such as data frames. Its robust integration with other Python libraries simplifies the data science workflow.
2. **NumPy:** Offers efficient operations on numerical data, enabling optimized performance for mathematical computations.
3. **Matplotlib:** Allows the creation of high-quality static visualizations, essential for clear and effective data representation.

These libraries were chosen because they are well-established in the data science community, offering reliability, extensive documentation, and a wide range of functionalities. While alternatives like R's ggplot2 or Tableau were considered, Python's cohesive ecosystem and the ease of integrating these libraries with our backend provided a clear advantage.

5. IDE Framework

- Build the IDE using the Monaco Editor (the foundation of VS Code) for a robust code-editing experience.
- Incorporate GUI components like drag-and-drop interfaces for dataset uploads and parameter tuning.

Optimization for Performance

1. Model Optimization:

- Pre-trained AI models were optimized using **ONNX Runtime** to enhance execution speed and reduce computational overhead.

2. Caching and Preprocessing:

Frequently used analysis results are cached to minimize redundant computations, improving the platform's overall efficiency.

3. Cross-Platform Packaging:

- The platform was packaged as a standalone application using **PyInstaller** and **Electron-packager**, eliminating the need for users to install dependencies.

Collaboration and Version Control

1. Git for Version Control

- We used **Git** for version control and collaboration due to its:
 1. **Distributed Nature:** Each team member could work independently and merge changes seamlessly.
 2. **Collaboration Features:** Git supports branching and pull requests, making it easy to manage contributions from multiple developers.
 3. **Industry Standard:** Git is widely adopted in the software development industry, ensuring familiarity and compatibility with third-party tools like GitHub.

2. Open-Source Contributions

- By utilizing open-source libraries and frameworks, the platform benefits from established functionalities while promoting community engagement.

UML Diagram

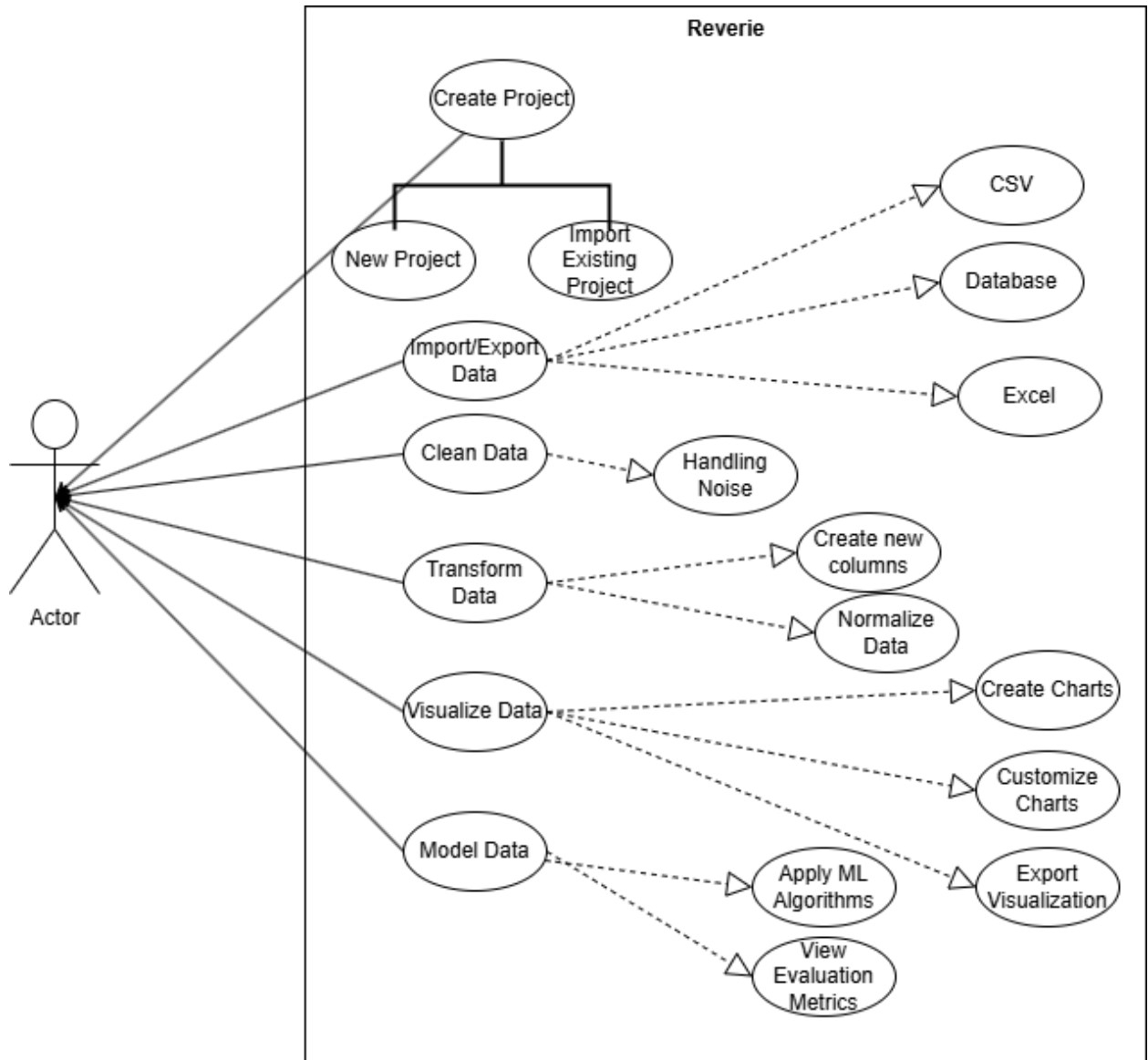
In the development of this project, the Unified Modeling Language (UML) served as a vital tool for visualizing, designing, and communicating the various aspects of the software solution. This section delves into the specific application of UML within the context of the project, highlighting its role in shaping the development process and guiding design decisions.

UML Utilization:

Throughout the project lifecycle, UML played a central role in capturing and representing key aspects of the software system. Here's how UML was utilized:

1. Use Case Diagrams:

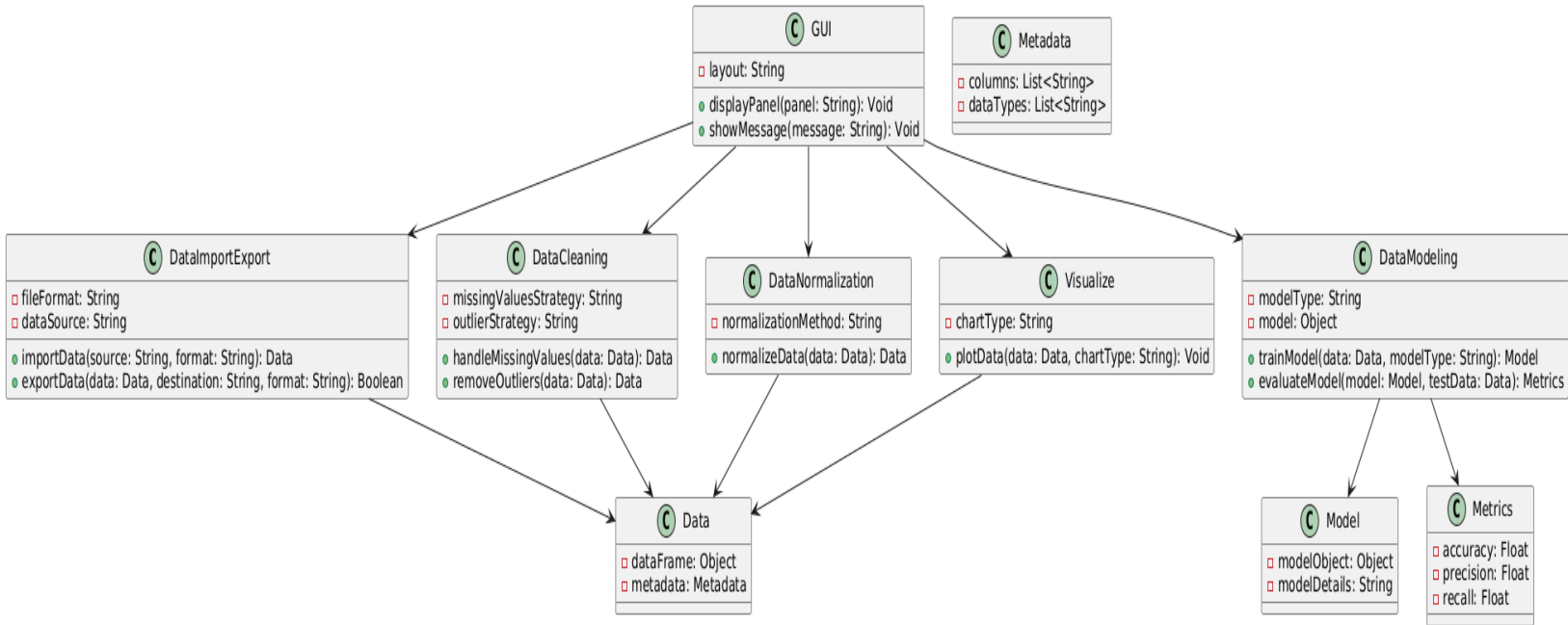
Use case diagrams were instrumental in capturing the functional requirements of the system from the perspective of users. These diagrams provided a visual representation of the various interactions between users and the system.



2. Class Diagrams:

Class diagrams were used to model the static structure of the system, including the entities, attributes, and relationships that comprise the software solution. By defining the

classes and their associations, class diagrams helped in organizing and structuring the system architecture, laying the groundwork for implementation.



TESTING PLANS

To ensure the reliability, usability, and overall quality of the software, a comprehensive testing strategy was developed and implemented. The testing phase was divided into three levels: unit testing, integration testing, and user testing. Each level targeted specific aspects of the system to ensure a robust and error-free experience for the end user.

Unit Testing

Objective

Unit testing focuses on testing individual components and modules of the software in isolation to ensure each operates as expected.

Scope

Frontend Testing:

- **Purpose:** Ensure individual graphical components function correctly and provide a responsive user experience.
- **Tests Conducted:**
 - Button functionality: Verify buttons like "Add Row," "Upload Dataset," and "Apply Preset" trigger appropriate actions.
 - Responsiveness: Test the GUI layout and performance on various screen sizes.
 - Input fields: Validate input types for datasets, feature names, and model parameters to prevent invalid entries.
- **Example:** Clicking the "Visualize Data" button loads the correct visualization options.

Backend Testing:

- **Purpose:** Validate the correct execution of backend logic and algorithms.
- **Tests Conducted:**

- Data processing functions: Ensure correct handling of missing values, duplicates, and outliers.
- Model selection algorithms: Validate the accuracy of the AI Assistant's model recommendations.
- API endpoints: Ensure proper communication with the frontend by simulating API calls.
- **Example:** Inputting a dataset with missing values triggers the correct imputation strategy based on user settings.

AI Assistant Testing:

- **Purpose:** Ensure the assistant provides accurate recommendations and insights.
- **Tests Conducted:**
 - **Dataset inspection:** Verify accurate detection of missing values, feature types, and outliers.
 - **Model suggestion:** Ensure recommended algorithms align with dataset characteristics.
- **Example:** The AI Assistant suggests logistic regression for a binary classification dataset and provides reasoning for the choice.

Tools Used:

- **PyTest:** For Python-based backend testing.
- **Jest:** For unit testing Electron.js frontend components.

Integration Testing

Objective

Integration testing ensures that individual components work together seamlessly and data flows correctly between modules.

Scope

- **Frontend-Backend Communication:**

- **Tests Conducted:**

- Validate the frontend can send user inputs (e.g., dataset selection, settings) to the backend and retrieve processed results.
 - Ensure responses from the backend, such as cleaned datasets or AI suggestions, are correctly displayed in the GUI.

- **Example:** A dataset uploaded in the GUI is processed by the backend, with results such as missing value counts displayed in real-time.

- **Module Interaction:**

- **Tests Conducted:**

- Test the integration of presets with the cleaning and modeling modules.
 - Verify the settings page correctly updates parameters in other modules, such as default cleaning techniques or visualization preferences.

- **Example:** Changing the data cleaning method in settings updates the process used in the “Clean Data” module.

Tools Used:

- Postman: For testing API communication.
- Selenium: For automating end-to-end workflows in the GUI.

User Testing

Objective

User testing focuses on evaluating the software’s usability, intuitiveness, and overall effectiveness from the perspective of the target audience.

Approach:

- **Participant Selection:**

- Diverse participants with varying levels of technical expertise, ranging from data science professionals to non-technical users.

- **Tasks Given:**

- Upload a dataset and clean it using predefined presets.
- Visualize the data and interpret the insights provided by the software.
- Build a simple model using AI Assistant recommendations.

Key Areas Assessed:

- **Ease of Use:**

- How intuitive the software is for new users.
- Time taken to complete common tasks like uploading datasets or visualizing data.

- **Accuracy:**

- Users evaluate the AI Assistant's recommendations for cleaning methods and model selection.

- **Efficiency:**

- Assess how effectively the system improves workflows by reducing manual coding.

CODING PRACTICES

In the development of the software, adherence to **consistent** code style standards was paramount to ensure readability, maintainability, and **collaboration** among team members. The project followed a set of established guidelines derived from industry **best practices** and the preferences of the development team. The code style guidelines covered various aspects, including but not limited to:

Naming Conventions:

- Meaningful and descriptive names for variables, functions, classes, and other identifiers were used to enhance readability and understanding.
- **CamelCase** naming convention for class names, and camelCase or **snake_case** for variables and functions.

Formatting:

- Consistent indentation with spaces or tabs, adhering to the chosen indentation style.
- Proper spacing around operators and between code elements for improved readability.
- Clear and concise comments to document code functionality, especially for complex or critical sections.

Code Organization:

- Logical grouping of related functions and classes within modules or files.
- Separation of concerns, with clear distinctions between presentation (HTML/CSS), business logic (JavaScript), and data access layers (Database queries).

Documentation:

- Comprehensive documentation of code modules, including high-level overviews, function descriptions, parameter details, return types, and usage examples.
- Integration of docstrings for Python functions and classes, following the PEP 257 guidelines.

Version Control:

- Effective utilization of version control systems such as Git for collaborative development, including branching strategies, commit message conventions, and pull request reviews.

CONCLUSION

In this report, we have explored the significant challenge faced by professionals in industries with abundant data but limited technical expertise in utilizing advanced data science tools. Despite the increasing availability of data, the gap between the data and the ability to analyze it has hindered decision-making and insights across many sectors. Traditional data science tools, while powerful, require a level of programming knowledge that many professionals lack.

To address this issue, we introduced **Reverie**, an intuitive software platform designed to simplify and automate key data science tasks, making them accessible to users with little to no technical experience. By offering functionalities like easy data import, preprocessing, visualizations, and machine learning model building, **Reverie** empowers users to analyze data and generate insights independently. This platform reduces the reliance on external consultants, thus streamlining decision-making processes and improving overall productivity.

The development of **Reverie** represents a crucial step in democratizing data science, enabling a broader range of professionals to harness the power of data to drive informed decisions. Moving forward, further refinement of the platform and incorporation of additional features will enhance its usability, ensuring that data-driven insights are available to all, regardless of technical background.

REFERENCES

- [1] IBM, “Data science,” *Ibm.com*, Sep. 21, 2021. <https://www.ibm.com/think/topics/data-science>
- [2] R. K. K, “Data Science in the 21st Century: Evolution, Challenges, and Future Directions,” *vol. 1, no. 1*, pp. 1-01, Jan. 2024. Available: <https://www.researchgate.net/publication/377598352>.
- [3] N. Kumar, “Challenges and Opportunities of Data Science,” *Analytics Insight*, Jul. 08, 2023. <https://www.analyticsinsight.net/data-science/challenges-and-opportunities-of-data-science>
- [4] M. Ghasemaghaei and G. Calic, “Does big data enhance firm innovation competency? The mediating role of data-driven insights,” *Journal of Business Research*, vol. 104, pp. 69–84, Nov. 2019, doi: <https://doi.org/10.1016/j.jbusres.2019.07.006>.
- [5] R. M. Parthe, “Comparative Analysis of Data Visualization Tools: Power BI and Tableau,” *Indian Scientific Journal Of Research In Engineering And Management*, vol. 07, no. 10, pp. 1–11, Oct. 2023, doi: <https://doi.org/10.55041/ijsrem26272>.
- [6] U. Kar, “What are Power BI disadvantages and limitations?,” *www.knowledgehut.com*, Jul. 04, 2024. <https://www.knowledgehut.com/blog/business-intelligence-and-visualization/power-bi-limitations>
- [7] T. De Bie, L. De Raedt, J. Hernández-Orallo, H. H. Hoos, P. Smyth, and C. K. I. Williams, “Automating data science,” *Communications of the ACM*, vol. 65, no. 3, pp. 76–87, Mar. 2022, doi: <https://doi.org/10.1145/3495256>
- [8] J. D. Shaikh, “Automation in data science: Simplifying repetitive tasks - Jhonathan David Shaikh,” *Jhonathan David Shaikh*, Nov. 14, 2023. <https://jhonathands.com/automation-in-data-science-simplifying-repetitive-tasks/>
- [9] K. Patel, “AutoML and Automated Data Science by Democratizing AI through End-to-End Automation,” *International Journal for Research in Applied Science and*

Engineering Technology, vol. 12, no. 10, pp. 494–504, Oct. 2024, doi:
<https://doi.org/10.22214/ijraset.2024.64555>

- [10] Amruta Meshram, “Empowering Non-Technical Users with Data Science Tools: A Roadmap to Democratizing Data,” *Medium*, Dec. 13, 2023.
<https://medium.com/@amruta1006/empowering-non-technical-users-with-data-science-tools-a-roadmap-to-democratizing-data-89f240c1386c>
- [11] P. Devi and Kishori Lal Bansal, “Data science in healthcare: techniques, challenges and opportunities,” *Health and technology*, Apr. 2024, doi:
<https://doi.org/10.1007/s12553-024-00861-8>
- [12] Innovare, “Data Science in Education: Everything You Need to Know,” *Innovare | Social Innovation Partners*, Sep. 18, 2023. <https://innovaresip.com/resources/blog/data-science-in-education-guide/>